Notes of parallelism and concurrency in software

Dario Marcone

Jeudi 19 février 2026

Contents

1 Metrics **2**

1 Metrics

Iron Law

$$\text{Processor Performance} = \frac{\text{Instructions}}{\text{Program}} \cdot \frac{\text{Cycles}}{\text{Instruction}} \cdot \frac{\text{Seconds}}{\text{Cycle}}.$$

-

- $\mathbf{\frac{Instructions}{Program}}$:

  The $\frac{\text{Instructions}}{\text{Program}}$ part represent the number of lines of code of your program, it is influenced by:

  - The compiler: depends of how many machine instructions per line of code.
  - The runtime system: depends of the language interpreted, the runtime overhead, etc

  > *Remark* The runtime system is a program given with the programmation language (ex. JVM for Java), that gives necessary services and mecanisms to the program during its execution, like memory alllocation, etc...

- $\mathbf{\frac{Cycles}{Instruction}}$:

  The $\frac{\text{Cycles}}{\text{Instruction}}$ part represent the number of cycles by instructions, it is influenced by:

  - The front-end cycles (instruction cache, branch predictor, etc...),
  - The back-end cycles (data cache, ALU, FPU, etc...),
  - The memory (latency, bandwith).

  > *Remark* Front-end part of the pipeline CPU is the instruction fetching, decoding and branch prediction part, and the banc-end is the computation, memory access and data stockage part.

- $\mathbf{\frac{Seconds}{Cycle}}$:

  The $\frac{\text{Seconds}}{\text{Cycle}}$ part is the clock frequency. We have to be careful! The cycles per instruction (CPI) is a good metric for uniprocessor, but not necessarly for multi processor. In multiprocessors, some core may have to wait for others, to have a specific information or whatever, so when the core wait, it will just do some spinning threads which is very fast $\implies$ a low CPI! But in fact, you dont execute the program particularely faster.
  It is better to use some metric for " useful work", like:

  - Transactions complete per second,
  - Request services per second.

Vocabulary parenthesis:

- **Latency:** time it takes for an event to complete
- **Bandwidth:** how many events per unit time

Reducing latency increases bandwidth but not vice versa.

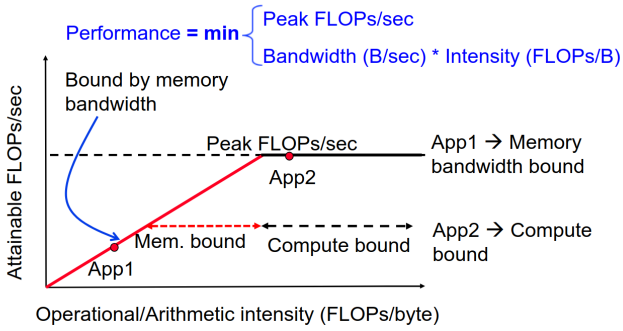
Amdahl's law

The amdahl's law shows that we have to improve efficency of what take the most of the time in our code execution.

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}.$$

-

Roofline Model



- The x-axis correspond to the number of data used for computation: if it is low, then it means you load/write a lot of data but you don't make computation with them. You are limited by the speed of loading/writing of/in the memory, not by the CPU calculation speed $\implies$ memory bound.
- The y-axis correspond to the number of FLOPs that you can reach on the CPU, if you are doing a lot of computation by byte, then the CPU reach its maximum calculation speed $\implies$ peak FLOPs/sec, independent of memory.
- We can know by what we are limited by taking the minimum of

  $$\text{Performance} = \begin{cases} \text{Peak FLOPs/sec} \\ \text{Bandwidth} \cdot \text{Intensity} \end{cases}$$

  It means that

  if $B \cdot \text{Intensity} < \text{Peak FLOPs/sec} \implies$ we are limited by memory,

  if $B \cdot \text{Intensity} \geq \text{Peak FLOPs/sec} \implies$ we are limited by CPU.

Parallel efficiency

$$\text{Parallel efficiency} = \frac{\text{Speedup}}{\#\text{ of cores}}.$$

-

Vocabulary parenthesis:

- **Speedup(n)**: performance improvement of n cores over 1 core,
- **Costup(n)**: cost increase of n cores over 1 core.

Different type of mean

- Arithmetic mean:

$$\frac{1}{n}\sum_{i=1}^{n}\text{Time}_i,$$

- Harmonic mean:

$$\frac{n}{\sum_{i=1}^{n}\frac{1}{\text{Rate}_i}},$$

- Geometric mean:

$$\text{Relative Rate} = \frac{\text{Rate}}{\text{Rate}_{\text{ref}}} = \frac{\text{Time}_{\text{ref}}}{\text{Time}},$$

- Geometric mean:

$$\sqrt[n]{\prod_{i=1}^{n}\text{Relative Rate}_i} = \frac{\sqrt[n]{\prod_{i=1}^{n}\text{Rate}_i}}{\text{Rate}_{\text{ref}}}.$$